

CÔNG NGHỆ PHẦN MỀM - THIẾT KẾ KIẾN TRÚC HỆ THỐNG

BIÊN BẢN THẢO LUẬN NHÓM

BÀI TẬP KIẾN TRÚC PHẦN MỀM

Đề tài thảo luận: Hệ thống Bán vé Tàu Cao tốc Trực tuyến (ATIS)

Môn học: Công nghệ Phần mềm - Thiết kế kiến trúc hệ thống

Nhóm	Voz
Thành viên	Võ Trịnh Quốc Huy Nguyễn Thành Trung Võ Văn Khôi Nguyễn Phạm Nguyễn Tuấn Kiệt
Nội dung	Lựa chọn kiến trúc kết hợp và giải quyết xung đột giữa 15.000 TPS với PCI-DSS

1. Tóm tắt yêu cầu của đề bài

ATIS là hệ thống bán vé tàu cao tốc trực tuyến phải chịu tải rất lớn trong dịp Tết, với mục tiêu 15.000 giao dịch mỗi giây (TPS). Hệ thống đồng thời tiếp nhận thông tin thanh toán thẻ nên phải đáp ứng PCI-DSS, bao gồm bảo vệ dữ liệu chủ thẻ, kiểm soát truy cập, ghi nhật ký và phân tách môi trường dữ liệu thẻ.

Đề bài yêu cầu hai đầu ra: (1) chọn một kiến trúc cấp cao kết hợp, trong đó nhóm chọn MVC cho tầng Presentation và Layered Architecture cho Backend; (2) phân tích và đề xuất cách giải quyết xung đột giữa hiệu năng cao và bảo mật thẻ tín dụng.

2. Cơ sở lý thuyết

2.1. Mẫu MVC

MVC tách giao diện thành Model, View và Controller. Trong ATIS, View hiển thị lịch tàu, giá vé và trạng thái đặt chỗ; Controller tiếp nhận yêu cầu HTTP/API; View Model/DTO mang dữ liệu đã được định dạng. Sự tách biệt này giúp thay đổi giao diện web/mobile mà không làm thay đổi quy tắc bán vé.

2.2. Layered Architecture

Backend được chia thành Application Layer, Domain Layer và Infrastructure Layer. Mỗi tầng chỉ đảm nhiệm một nhóm trách nhiệm và phụ thuộc theo hướng vào trong. Quy tắc giữ chỗ, tính giá và chống bán trùng thuộc Domain; Redis, cơ sở dữ liệu, message broker và cổng thanh toán thuộc Infrastructure.

2.3. Thuộc tính chất lượng và trade-off

Hiệu năng và bảo mật không phải hai lựa chọn loại trừ nhau, nhưng cùng tiêu thụ tài nguyên. Mã hóa, kiểm tra quyền, ghi audit và gọi cổng thanh toán làm tăng độ trễ; ngược lại, cache hoặc bất đồng bộ sai vị trí có thể tạo vé trùng, lộ dữ liệu hoặc trả kết quả thanh toán không chính xác. Do đó cần xác định luồng nào được cache/bất đồng bộ và luồng nào bắt buộc nhất quán chặt.

2.4. Phạm vi PCI-DSS

PCI-DSS áp dụng cho các thành phần lưu trữ, xử lý hoặc truyền dữ liệu chủ thẻ, và cả các thành phần có thể ảnh hưởng đến an toàn của môi trường đó. Phân đoạn mạng và tokenization giúp giảm phạm vi CDE, nhưng không thay thế các kiểm soát bắt buộc. Phiên bản tham chiếu của nhóm là PCI DSS v4.0.1 [3], [4].

3. Thảo luận nhiệm vụ 1 - Lựa chọn kiến trúc cấp cao kết hợp

3.1. Các yếu tố chi phối quyết định

- Tải đột biến 15.000 TPS, tập trung vào các khung giờ mở bán dịp Tết.
- Nghiệp vụ có tính cạnh tranh cao: nhiều người có thể đặt cùng một ghế trong cùng thời điểm.
- Giao diện web và mobile thay đổi thường xuyên, trong khi quy tắc giá và giữ chỗ cần ổn định.
- Thanh toán thẻ phải được cô lập, truy cập tối thiểu và có bằng chứng audit.
- Hệ thống cần mở rộng từng cụm chức năng mà không buộc nhân bản toàn bộ ứng dụng.

3.2. Phương án nhóm lựa chọn

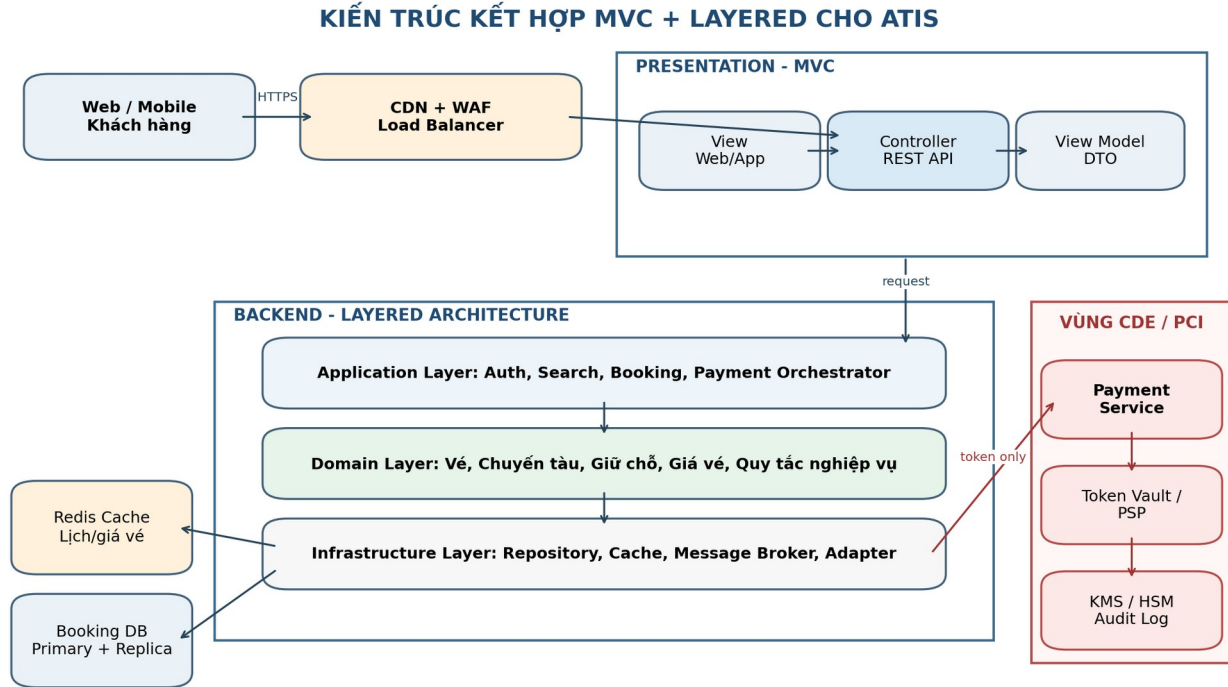
Nhóm thống nhất chọn MVC ở Presentation kết hợp Layered Architecture ở Backend. Đây là một kiến trúc kết hợp ở cấp cao: MVC giải quyết sự thay đổi và khả năng kiểm thử của giao diện; Layered kiểm soát phụ thuộc, cô lập nghiệp vụ và cho phép thay thế hạ tầng. Payment Service được đặt trong vùng CDE riêng, giao tiếp với phần còn lại bằng mã giao dịch và payment token.

Tầng/khối	Trách nhiệm chính	Quy tắc phụ thuộc
Presentation - MVC	View web/mobile, Controller/API, xác thực phiên, validation định dạng.	Không chứa quy tắc giá hoặc thao tác trực tiếp CSDL.
Application Layer	Điều phối Search, Booking, Payment; transaction boundary; idempotency.	Gọi Domain; dùng interface cho hạ tầng.
Domain Layer	Chuyến tàu, vé, ghế, giữ chỗ, giá, chính sách hoàn/hủy.	Không phụ thuộc framework, DB hay cổng thanh toán.
Infrastructure Layer	Repository, Redis, message broker, adapter PSP, logging.	Hiện thực interface do tầng trong định nghĩa.
Payment/CDE	Tokenization, ủy quyền thanh toán, KMS/HSM, audit bảo mật.	Mạng tách biệt; ATIS chỉ nhận token và trạng thái.

3.3. Vì sao không chọn một mẫu đơn lẻ?

Chỉ dùng MVC cho toàn hệ thống dễ khiến Controller phình to, nghiệp vụ thanh toán và đặt chỗ trộn với xử lý HTTP. Chỉ dùng Layered lại chưa mô tả rõ cách tổ chức nhiều giao diện. Kết hợp hai mẫu cho phép mỗi mẫu giải quyết đúng vấn đề, đồng thời vẫn giữ một hướng phụ thuộc rõ ràng.

4. Sơ đồ kiến trúc đề xuất



Luồng chính có thể mở rộng ngang; dữ liệu thẻ được cô lập khỏi phần lớn hệ thống ATIS.

Hình 1. Kiến trúc kết hợp MVC + Layered và vùng thanh toán PCI của ATIS

Luồng truy cập đi qua CDN/WAF và Load Balancer trước khi vào các Controller stateless. Backend điều phối nghiệp vụ theo các tầng. Cache và read replica phục vụ dữ liệu đọc nhiều; Booking DB xử lý giữ chỗ; Payment Service nằm trong CDE tách biệt. Message broker thuộc Infrastructure để chuyển các tác vụ hậu xử lý, không dùng để trì hoãn quyết định thanh toán hoặc xác nhận ghế.

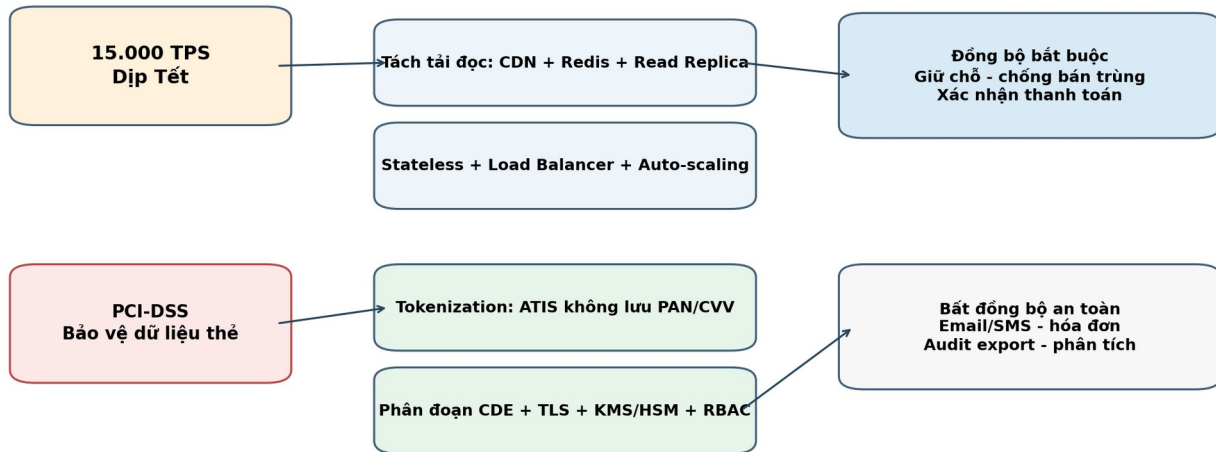
5. Phân tích nhiệm vụ 2 - Xung đột giữa 15.000 TPS và PCI-DSS

5.1. Biểu hiện của xung đột

Nhu cầu hiệu năng	Nguy cơ nếu tối ưu sai	Yêu cầu bảo mật liên quan
Cache mạnh	Cache PAN/CVV hoặc response nhạy cảm; dữ liệu cũ về ghế.	Giảm lưu dữ liệu thẻ; giới hạn dữ liệu nhạy cảm.
Bất đồng bộ	Trả thành công trước khi PSP xác nhận; khó đối soát.	Toàn vẹn giao dịch và audit đầy đủ.
Scale-out	Tăng số nút, secret và bề mặt tấn công.	Cấu hình chuẩn, quản lý khóa, kiểm soát truy cập.
Giảm bước kiểm tra	Bỏ WAF, xác thực, logging hoặc mã hóa để giảm latency.	Không được hy sinh kiểm soát PCI bắt buộc.
Ghi DB nhanh	Race condition làm bán trùng ghế; retry tạo thanh toán kép.	Tính toàn vẹn và truy vết giao dịch.

5.2. Nguyên tắc giải quyết

Nhóm ưu tiên bảo mật thẻ là ràng buộc bắt buộc, còn hiệu năng được đạt bằng cách giảm phạm vi PCI và tối ưu kiến trúc. Hệ thống không lưu PAN đầy đủ và tuyệt đối không lưu CVV sau ủy quyền. Giao diện thanh toán dùng hosted fields/redirect của nhà cung cấp thanh toán; ATIS chỉ nhận payment token. Nhờ vậy phần lớn tải tìm chuyến, xem giá và giữ chỗ không chạm dữ liệu thẻ.



Nguyên tắc: không bỏ kiểm soát PCI để đạt TPS; tối ưu phạm vi, đường đi dữ liệu và vị trí áp dụng kiểm soát.

Hình 2. Cách phân tách luồng để đồng thời đáp ứng hiệu năng và PCI-DSS

6. Giải pháp kỹ thuật chi tiết

6.1. Nhóm giải pháp đạt 15.000 TPS

- CDN phân phối tài nguyên tĩnh; WAF và rate limiting chặn lưu lượng bất thường trước cụm ứng dụng.
- Controller/API stateless chạy nhiều instance sau Load Balancer; auto-scaling theo RPS, độ trễ p95 và độ sâu hàng đợi.
- Redis cache lịch tàu, ga, giá tham khảo và availability snapshot; dùng TTL ngắn và invalidation theo sự kiện.
- Read replica cho truy vấn tra cứu; primary chỉ nhận các ghi giữ chỗ, phát hành vé và cập nhật trạng thái.
- Chỉ mục phù hợp cho train_id, departure_date, seat_id và booking_status; partition theo ngày chạy tàu.
- Message broker xử lý email/SMS, xuất audit, đồng bộ báo cáo và phát hành hóa đơn sau giao dịch.
- Connection pool, TLS session resumption và hardware acceleration giúp giảm chi phí bắt tay/mã hóa mà không hạ chuẩn bảo mật.

6.2. Nhóm giải pháp PCI-DSS

- Tokenization/hosted payment page: trình duyệt gửi dữ liệu thẻ trực tiếp đến PSP; ATIS chỉ giữ token và bốn số cuối khi thật sự cần.
- Phân đoạn mạng: CDE chỉ gồm Payment Service, token vault/PSP adapter, KMS/HSM và hệ thống audit có liên quan; mặc định deny giữa các vùng.
- TLS mạnh cho dữ liệu truyền; mã hóa dữ liệu nhạy cảm khi lưu; khóa tách khỏi dữ liệu và được quản lý/luân chuyển bằng KMS/HSM.
- RBAC và nguyên tắc tối thiểu; MFA cho tài khoản quản trị; secret ngắn hạn; tách nhiệm vụ vận hành và phê duyệt.

CÔNG NGHỆ PHẦN MỀM - THIẾT KẾ KIẾN TRÚC HỆ THỐNG

- Log không chứa PAN/CVV; masking trước khi ghi; log tập trung, chống sửa đổi, đồng bộ thời gian và có cảnh báo.
- Quét lỗ hổng, kiểm thử xâm nhập, quản lý bản vá, secure coding và kiểm soát thay đổi trước khi triển khai.

7. Phân loại đồng bộ, bất đồng bộ và nhất quán

Luồng	Cách xử lý	Lý do
Tìm chuyến / xem giá	Cache + read replica; eventual consistency có giới hạn.	Đọc rất nhiều, chấp nhận dữ liệu trễ vài giây nếu lúc đặt được kiểm tra lại.
Giữ chỗ	Đồng bộ; atomic update/unique constraint; thời hạn giữ chỗ.	Không được bán cùng một ghế cho hai người.
Tạo yêu cầu thanh toán	Đồng bộ đến lúc nhận trạng thái rõ ràng; idempotency key.	Retry không tạo hai giao dịch thanh toán.
Callback PSP	Xác minh chữ ký; cập nhật trạng thái bằng state machine.	Chống giả mạo và cập nhật sai thứ tự.
Email/SMS/hóa đơn	Bất đồng bộ qua broker; retry + dead-letter queue.	Không nằm trên critical path của phản hồi mua vé.
Phân tích/báo cáo	Event stream / bản sao dữ liệu.	Không tải truy vấn phân tích lên Booking DB chính.

8. Luồng mua vé an toàn và chịu tải

- Khách tìm chuyến. Controller ưu tiên dữ liệu cache; nếu cache miss thì đọc từ replica và ghi cache có TTL.
- Khách chọn ghế. Booking Service kiểm tra lại trạng thái tại primary và tạo seat hold nguyên tử với thời gian hết hạn.
- Hệ thống tạo booking_id và idempotency_key; không đưa số thẻ vào request nội bộ hoặc log.
- Hosted payment field gửi PAN/CVV trực tiếp đến PSP. Payment Service nhận token và gửi yêu cầu authorize qua kết nối TLS.
- PSP trả kết quả. Payment Service xác minh response, lưu trạng thái và phát sự kiện có chữ ký/định danh duy nhất.
- Booking Service chuyển trạng thái HOLD sang CONFIRMED khi thanh toán thành công; nếu thất bại hoặc timeout thì giải phóng ghế theo chính sách.
- Email, SMS, hóa đơn và analytics xử lý bất đồng bộ; giao diện trả kết quả mua vé ngay sau khi giao dịch cốt lõi hoàn tất.

9. Kế hoạch kiểm chứng 15.000 TPS

Con số 15.000 TPS chỉ được chấp nhận sau kiểm thử tải, không suy ra từ sơ đồ. Nhóm đề xuất mô phỏng đúng tỷ lệ tìm kiếm/giữ chỗ/thanh toán, tăng tải theo bậc và đo p95/p99, throughput, error rate, saturation, cache-hit ratio, DB lock wait, queue lag và thời gian phản hồi PSP.

Tiêu chí	Mục tiêu đề xuất	Hành động khi vi phạm
Throughput	≥ 15.000 TPS trong cửa sổ cao điểm	Scale-out; kiểm tra bottleneck WAF/LB/cache/DB.
Độ trễ	p95 theo SLO từng API; không gộp search với payment	Tách dashboard; tối ưu đúng critical path.
Lỗi	Không phát sinh bán trùng; lỗi có kiểm soát	Load shedding; circuit breaker; hàng đợi giới hạn.

CÔNG NGHỆ PHẦN MỀM - THIẾT KẾ KIẾN TRÚC HỆ THỐNG

Tiêu chí	Mục tiêu đề xuất	Hành động khi vi phạm
An toàn retry	Một idempotency key chỉ tạo một kết quả	Kiểm thử timeout, retry và callback lặp.
PCI evidence	Log đầy đủ nhưng không có PAN/CVV	Data-loss scan; kiểm tra masking và quyền truy cập.

10. Phân tích và phản biện phương án

MVC + Layered đem lại cấu trúc rõ, dễ học và phù hợp phạm vi bài tập. Nhược điểm là request có thể đi qua nhiều tầng và tạo boilerplate. Nhóm chấp nhận chi phí này ở mức hợp lý, đồng thời cấm tạo tầng chỉ để chuyển tiếp dữ liệu mà không có trách nhiệm rõ ràng.

Cache không phải nguồn sự thật cho việc bán ghế. Availability trên màn hình chỉ là ảnh chụp gần thời điểm hiện tại; quyết định cuối cùng phải được kiểm tra nguyên tử tại Booking DB. Tương tự, message broker không được dùng để trả thành công giả trước khi có kết quả thanh toán.

Việc thuê PSP và dùng tokenization giúp giảm phạm vi PCI nhưng không làm ATIS tự động tuân thủ. Trang web, script thanh toán, cấu hình mạng, tài khoản quản trị, log và quy trình vận hành vẫn có thể ảnh hưởng đến CDE nên vẫn phải được đánh giá theo phạm vi thực tế [3], [4].

11. Mã Mermaid để nhóm dựng lại sơ đồ

11.1. Component/Architecture Diagram

```
flowchart TB
    U[Web/Mobile] --> E[CDN + WAF + Load Balancer]
    E --> MVC[Presentation: MVC]
    MVC --> APP[Application Layer]
    APP --> DOM[Domain Layer]
    DOM --> INF[Infrastructure Layer]
    INF --> CACHE[(Redis Cache)]
    INF --> DB[(Booking DB + Read Replica)]
    INF --> MQ[(Message Broker)]
    APP -->|payment token| PAY[Payment Service - CDE]
    PAY --> PSP[Token Vault / PSP]
    PAY --> KMS[KMS / HSM + Audit]
```

11.2. Sequence Diagram cho luồng mua vé

```
sequenceDiagram
    actor C as Khách hàng
    participant B as Booking Service
    participant D as Booking DB
    participant P as Payment Service
    participant G as PSP
    C->>B: holdSeat(seatId, idempotencyKey)
    B->>D: atomicCreateHold()
    D-->>B: holdId + expiresAt
    C->>G: PAN/CVV qua hosted fields
    G->>P: paymentToken
    P->>G: authorize(token, amount)
    G-->>P: approved / declined
    P-->>B: signedPaymentResult
    alt approved
        B->>D: confirmBooking(holdId)
    else declined/timeout
        B->>D: releaseHold(holdId)
    end
    end
```

12. Kết luận của nhóm

Nhóm chọn MVC cho Presentation và Layered Architecture cho Backend của ATIS. Kiến trúc này tách rõ giao diện, điều phối ứng dụng, quy tắc nghiệp vụ và hạ tầng; Payment Service được cô lập trong vùng CDE. Đây là phương án cân bằng giữa khả năng thay đổi giao diện, kiểm soát phụ thuộc và mở rộng theo tải.

Xung đột giữa 15.000 TPS và PCI-DSS được giải quyết bằng giảm phạm vi dữ liệu thẻ, tokenization, phân đoạn CDE và tối ưu các đường đi không nhạy cảm bằng CDN, cache, read replica, scale-out và xử lý bất đồng bộ. Các luồng giữ chỗ và xác nhận thanh toán vẫn phải đồng bộ, idempotent và có audit. Không có kiểm soát PCI bắt buộc nào bị loại bỏ để đổi lấy hiệu năng.

13. Tài liệu tham khảo

- [1] Đại học Tôn Đức Thắng - Khoa Công nghệ Thông tin. Tài liệu học phần Công nghệ Phần mềm: Thiết kế kiến trúc hệ thống.
- [2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- [3] PCI Security Standards Council. PCI Data Security Standard (PCI DSS) v4.0.1, Document Library.
https://www.pcisecuritystandards.org/document_library/
- [4] PCI Security Standards Council. Glossary: CDE and Segmentation.
<https://www.pcisecuritystandards.org/glossary/>
- [5] PCI Security Standards Council. PCI DSS Scoping and Segmentation Guidance for Modern Network Architectures (2024). https://www.pcisecuritystandards.org/documents/Guidance-PCI-DSS-Scoping-and-Segmentation_v1.pdf